

# CIS 5521 – Homework 1

Md. Mizanur Rahman  
md.mizanur.rahman@temple.edu

September 9, 2026

## Problem 1

**a) Input:** Given an sorted array let say  $A$  of size  $n$  elements and a value  $x$ .

For example:  $A = [2, 5, 7, 8, 12, 16, 18]$  and  $x = 12$

**Output:** The position of the value  $x$  in the array if present, otherwise return "not found" or '-1'.

Example: For our given problem position of the value  $x = 12$  is  $5$  considering index starting from  $1$ .

**Relationship:** If the value  $x$  found in the array then the return value is  $i$  based on the relation  $A[i] = x$ , otherwise the algorithm should return 'not found' or '-1'.

## b) Iterative Binary Search:

---

### Algorithm 1 Iterative Binary Search

---

```
1: procedure BINARY-SEARCH-ITERATIVE( $A, x$ )
2:    $low = 1$ 
3:    $high = A.length$ 
4:   while  $low \leq high$  do
5:      $mid = \lfloor (low + high)/2 \rfloor$ 
6:     if  $A[mid] == x$  then
7:       return  $mid$ 
8:     else if  $x > A[mid]$  then
9:        $low = mid + 1$ 
10:    else
11:       $high = mid - 1$ 
12:    end if
13:  end while
14:  return  $-1$ 
15: end procedure
```

---

**Recursive Binary Search:****Algorithm 2** Recursive Binary Search

---

```

procedure BINARY-SEARCH-RECURSIVE( $A, x, low, high$ )
  if  $low > high$  then
    return  $-1$ 
  end if
   $mid = \lfloor (low + high)/2 \rfloor$ 
  if  $A[mid] == x$  then
    return  $mid$ 
  else if  $x > A[mid]$  then
    return BINARY-SEARCH-RECURSIVE( $A, x, mid + 1, high$ )
  else
    return BINARY-SEARCH-RECURSIVE( $A, x, low, mid - 1$ )
  end if
end procedure

```

---

c)

|    | BINARY-SEARCH( $A, x$ )                                 | <i>cost</i> | <i>times</i> |
|----|---------------------------------------------------------|-------------|--------------|
| 1  | $low = 1$                                               | $c_1$       | 1            |
| 2  | $high = A.length$                                       | $c_2$       | 1            |
| 3  | <b>while</b> $low \leq high$                            | $c_3$       | $k + 1$      |
| 4  | $mid = \left\lfloor \frac{low + high}{2} \right\rfloor$ | $c_4$       | $k$          |
| 5  | <b>if</b> $A[mid] == x$                                 | $c_5$       | $k$          |
| 6  | <b>return</b> $mid$                                     | $c_6$       | 1            |
| 7  | <b>else if</b> $A[mid] < x$                             | $c_7$       | $k$          |
| 8  | $low = mid + 1$                                         | $c_8$       | $r$          |
| 9  | <b>else</b>                                             | 0           | $k$          |
| 10 | $high = mid - 1$                                        | $c_{10}$    | $k - r$      |
| 11 | <b>return</b> $-1$                                      | $c_{11}$    | 1            |

So the running time is

$$T(n) = c_1 + c_2 + c_3(k + 1) + c_4k + c_5k + c_6 + c_7k + c_8r + c_{10}(k - r) + c_{11}.$$

Rearranging,

$$\begin{aligned}
 T(n) &= (c_3 + c_4 + c_5 + c_7 + c_{10})k \\
 &\quad + (c_8 + c_{10})r \\
 &\quad + (c_1 + c_2 + c_3 + c_6 + c_{11}).
 \end{aligned}$$

Again,

$$0 \leq r \leq k,$$

so the running time is proportional to  $k$ .

Because each recursive call reduces the problem size to approximately half of its previous size,

$$n \rightarrow \frac{n}{2} \rightarrow \frac{n}{4} \rightarrow \frac{n}{8} \rightarrow \cdots \rightarrow 1.$$

Therefore,

$$n/2^k \approx 1$$

$$2^k \approx n$$

$$k \approx \log_2 n$$

$$k = \Theta(\log n),$$

and hence,

$$\boxed{T_{\text{worst}}(n) = \Theta(\log n)}.$$

d) For recursive binary search, we can analyse it using recurrence.

$$T(n) = \begin{cases} \Theta(1), & \text{if } n \leq 1, \\ T(\lfloor n/2 \rfloor) + \Theta(1), & \text{if } n > 1. \end{cases}$$

This is the case because when there is 1 elements, it will take constant time by returning whether the value is present or not in the array. Otherwise, for every call the number of elements will be reduced to  $n/2$ . Therefore, the problem will be reduced by half.

$$\begin{aligned} T(n) &\leq d \log_2(n/2) + c \\ &= d(\log_2 n - \log_2 2) + c \\ &= d \log_2 n - d + c \\ &\leq d \log_2 n, \end{aligned}$$

where the last inequality holds as long as  $d \geq c$  and from  
Therefore,

$$T(n) = O(\log n).$$

In the worst case, the search interval is repeatedly divided by two:

$$n \rightarrow \frac{n}{2} \rightarrow \frac{n}{4} \rightarrow \frac{n}{8} \rightarrow \cdots \rightarrow 1.$$

The recursion terminates when

$$\frac{n}{2^k} = 1.$$

Thus,

$$2^k = n,$$

and hence

$$k = \log_2 n.$$

Therefore, the number of recursive calls is proportional to  $\log n$ , and the worst-case running time is

$$T(n) = O(\log n).$$

## Problem 2

**a) Insertion Sort:** For insertion sort, I selected number of comparisons and shifts as the indicator of time cost for the problem. Because, both shifts and comparisons are performed in the inner loop which constitute the dominant work of the problem.

**Merge Sort:** For merge sort, I selected number of comparisons and copies as the indicator because merge sort works by dividing the problem into two parts and then compare numbers between these two arrays and assign to sorted array. Thus, both comparison and copying constitute the dominant work.

**b) Sample Input:** 462, 513, 935, 91, 642, 952, 148, 84, 896, 558

Insertion Sort Output: 84, 91, 148, 462, 513, 558, 642, 896, 935, 952

Merge Sort Output: 84, 91, 148, 462, 513, 558, 642, 896, 935, 952

Considering the same input for both insertion and merge sort, the number of operations each of those algorithms take are given below:

For  $n = 100$  total operation taken:

insertion sort =  $2571 + 2571 = 5142$

merge sort =  $540 + 870 = 1410$

For  $n = 500$  total operation taken:

insertion sort:  $59679 + 59679 = 119358$

merge sort:  $3877 + 5486 = 9363$

For  $n = 1000$  total operation taken:

insertion sort:  $251804 + 251804 = 503608$

merge sort:  $8713 + 8713 = 17426$

For  $n = 1500$  total operation taken:

insertion sort:  $565553 + 565553 = 1131106$

merge sort:  $13935 + 18950 = 32885$

For  $n = 3000$  total operation taken:

insertion sort:  $2301238 + 2301238 = 4602476$

merge sort:  $30930 + 40902 = 71832$

For  $n = 6000$  total operation taken:

insertion sort:  $9029879 + 9029879 = 18059758$

merge sort:  $67847 + 87806 = 155653$

Several inputs ranging from 100 to 6000 were taken so that the growth of the algorithm observed with the increase of the input. Some inputs were also taken double from the previous input so that we can observe whether the output increased in 4 times than the previous output for merge sort and 2.2 times for merge sort since both the algorithms complexity are  $O(n^2)$  and

$O(n \log n)$  respectively. Small input alone would not clearly demonstrate the difference between the complexities of these two algorithms.

c) Comparison Table for different size of inputs and their corresponding outputs are given below:

Table 1: Comparison of Insertion and Merge Sort

| Input Size | insertComparisons | insertShifts | Total Insertion operations | mergeComparisons | mergcopies | Total Merge Operations |
|------------|-------------------|--------------|----------------------------|------------------|------------|------------------------|
| 100        | 2571              | 2571         | 5142                       | 540              | 870        | 1410                   |
| 500        | 59679             | 59679        | 119358                     | 3877             | 5486       | 9363                   |
| 1000       | 251804            | 251804       | 503608                     | 8713             | 8713       | 17426                  |
| 1500       | 565553            | 565553       | 1131106                    | 13935            | 18950      | 32885                  |
| 3000       | 2301238           | 2301238      | 4602476                    | 30930            | 40902      | 71832                  |
| 6000       | 9029879           | 9029879      | 18059758                   | 67847            | 87806      | 155653                 |

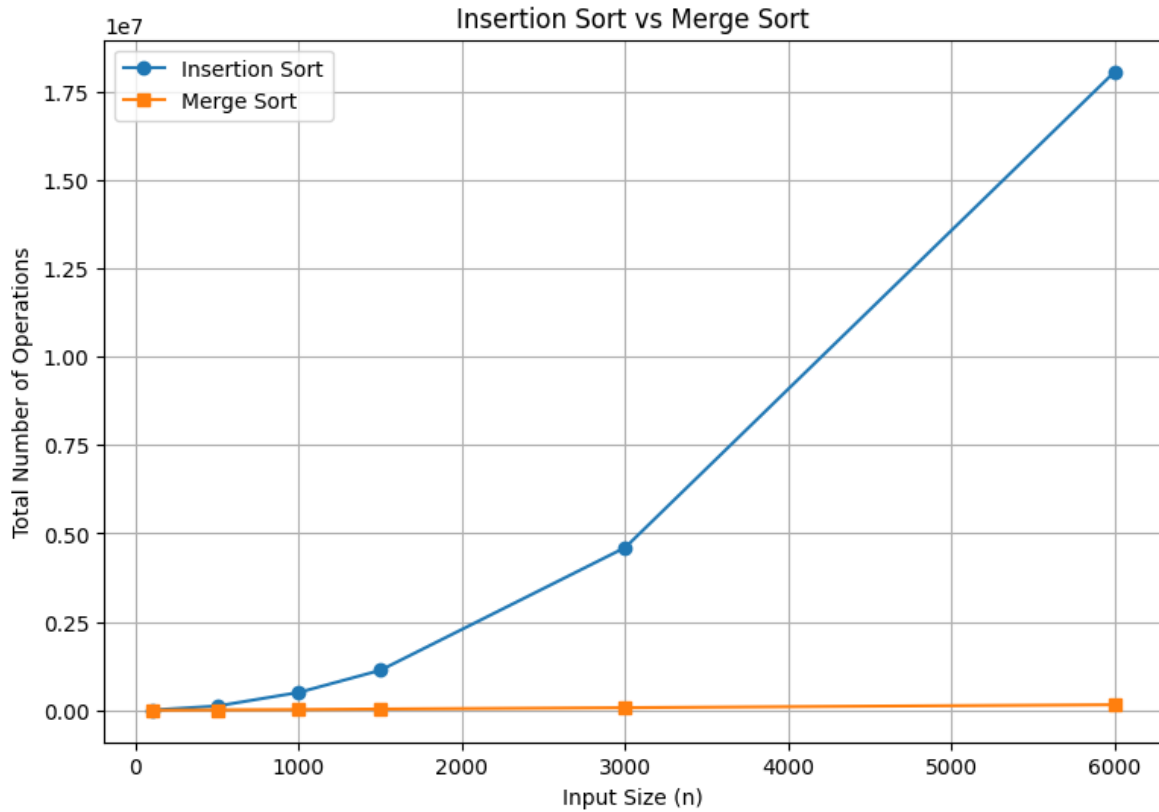


Figure 1: Number of operations for Insertion and Merge Sort for different inputs.

The graph also demonstrates the difference in their growth rates. The Insertion Sort curve rises very rapidly as  $(n)$  increases, showing quadratic growth. In contrast, the Merge Sort curve remains much flatter because  $(n \log n)$  grows significantly more slowly than  $(n^2)$ .

d) The theoretical time complexity of Insertion Sort is  $\Theta(n^2)$  for the average case, while the time complexity of Merge Sort is  $\Theta(n \log n)$ .

The experimental results generally agree with these theoretical conclusions.

For Insertion Sort, when the input size approximately doubles, the number of operations increases by about four times. For example, when the input size increases from (1500) to (3000), the number of operations increases from [1,131,106] to [4,602,476]

The ratio is  $\frac{4,602,476}{1,131,106} \approx 4.07$ .]

Similarly, when the input size increases from (3000) to (6000),  $\frac{18,059,758}{4,602,476} \approx 3.92$ .

These values are close to (4), which is what is expected for a quadratic algorithm because  $(2n)^2 = 4n^2$ .

Therefore, the experimental results strongly support the theoretical ( $\Theta(n^2)$ ) time complexity of Insertion Sort.

For Merge Sort, the number of operations grows much more slowly. When the input size increases from (1500) to (3000), the number of operations increases from (32,885) to (71,832):  $\frac{71,832}{32,885} \approx 2.18$ .

When the input size increases from (3000) to (6000), the number of operations increases from (71,832) to (155,653):  $\frac{155,653}{71,832} \approx 2.17$ .

Thus, when the input size doubles, the number of Merge Sort operations increases by only slightly more than two times. This behavior is consistent with the theoretical complexity:  $\Theta(n \log n)$ .

The difference between the two algorithms becomes especially clear for large input sizes. At (n=6000), Insertion Sort performs [18,059,758] operations, while Merge Sort performs only [155,653].]

Therefore,  $\frac{18,059,758}{155,653} \approx 116$ .

This means that, according to the operation counters used in this experiment, Insertion Sort performs approximately 116 times as many counted operations as Merge Sort for an input size of 6000.

The experiment don't follow the theoretical function exactly because we only considered the exact number of operations where asymptotic complexity describe the rate of growth of function.

Overall, the experimental results are consistent with the theoretical analysis: Insertion Sort:  $\Theta(n^2)$

Merge Sort:  $\Theta(n \log n)$

As the input size increases, Merge Sort becomes significantly more efficient than Insertion Sort, which agrees with the theoretical expectation.

### AI use Acknowledgment

Generative AI (ChatGPT) was used to assist with LaTeX formatting and presentation of this report.